

# Malware Disassembly and Code Analysis Using Ghidra

**EXPERIMENT NO: 10**

**DATE:**

## 1. AIM

To set up a secure analysis environment, utilize the Ghidra reverse engineering framework to disassemble and decompile binary code, and interpret low-level assembly functions to identify common malware behaviors, persistence mechanisms, and indicators of compromise.

## 2. SOFTWARE/TOOLS REQUIRED

- **Operating System:** Isolated Virtual Machine (Windows/Linux OS)
- **Forensics Tool:** Ghidra (Open-source software reverse engineering suite developed by the NSA)
- **Runtime Environment:** Java Development Kit (JDK) required for running Ghidra
- **Sample Files:** Benign sample binaries (benign\_sample1.bin, benign\_sample2.hex) or controlled test binaries exhibiting typical executable structures.

## 3. DESCRIPTION

Malware analysis is the critical process of dissecting compiled software to understand its capabilities, origins, and operational intent without executing it live (static analysis). Ghidra, an open-source Software Reverse Engineering framework developed by the National Security Agency (NSA), is utilized extensively by forensic investigators for this purpose. When original source code is unavailable or intentionally obfuscated by adversaries, analysts rely on Ghidra to bridge the gap between raw, machine-executable binaries and human-readable logic.

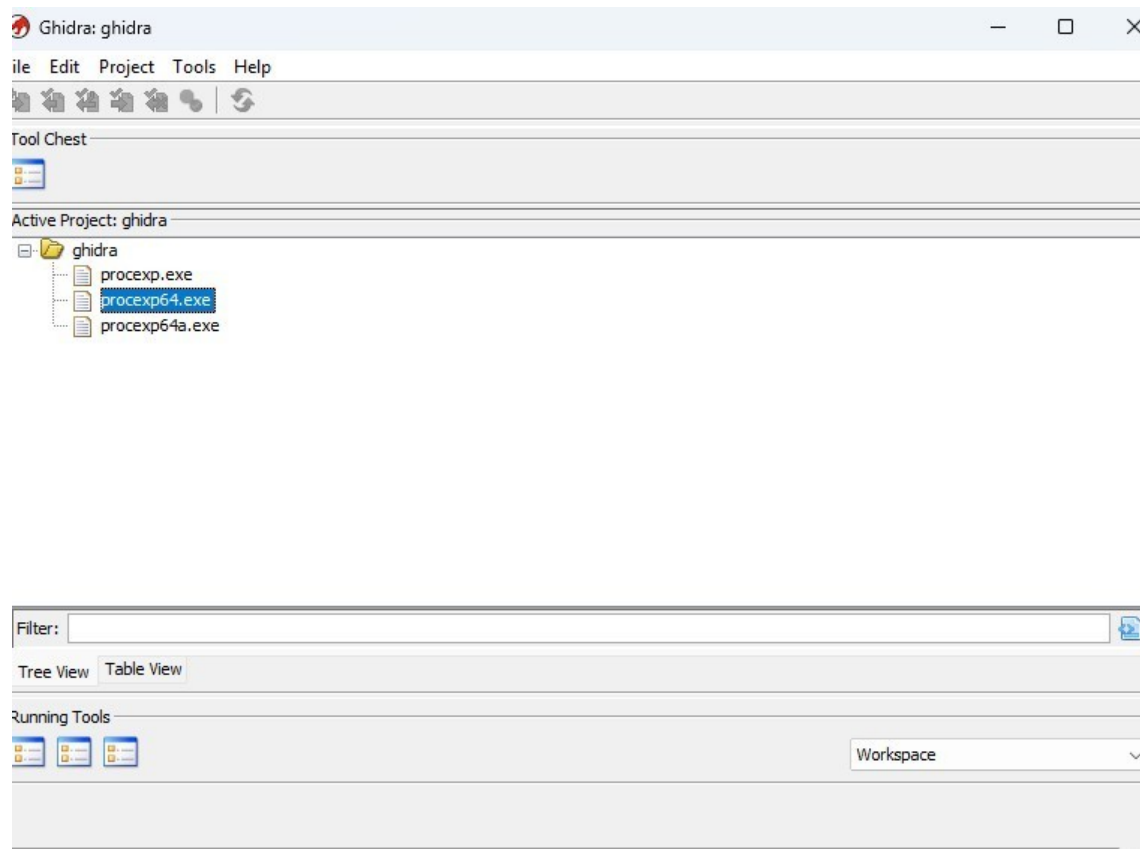
The framework's core strength lies in its CodeBrowser, which parses raw binary instructions into low-level assembly language, while its advanced Decompiler algorithmically reconstructs higher-level, C-like pseudo-code. This dual approach significantly reduces the cognitive load required to decipher complex malware algorithms. To further aid in understanding the execution logic, Ghidra generates interactive Control Flow Graphs (CFGs)

that visually map out function calls, decision branches, and loops, allowing analysts to trace execution paths and identify evasion techniques.

## 4. PROCEDURE

### Step1: Installation, Project Initialization, and Binary Import

1. Open the Kali Linux terminal and install the Ghidra framework using the advanced packaging tool:  
  
Bash: `sudo apt update && sudo apt install ghidra-y`
2. Launch Ghidra from the terminal by typing `ghidra`
3. create a new non-shared project titled `Ghidra_Malware_Analysis`.
4. Navigate to `File > Import File`, select the target binary (`/bin/ls`), and proceed with the default import configuration.



*Figure 1: Initial Ghidra Project Window showing the newly imported binary*

4. Double-click the imported file to open it in the Code Browser tool and execute the initial Auto-Analyze function when prompted.

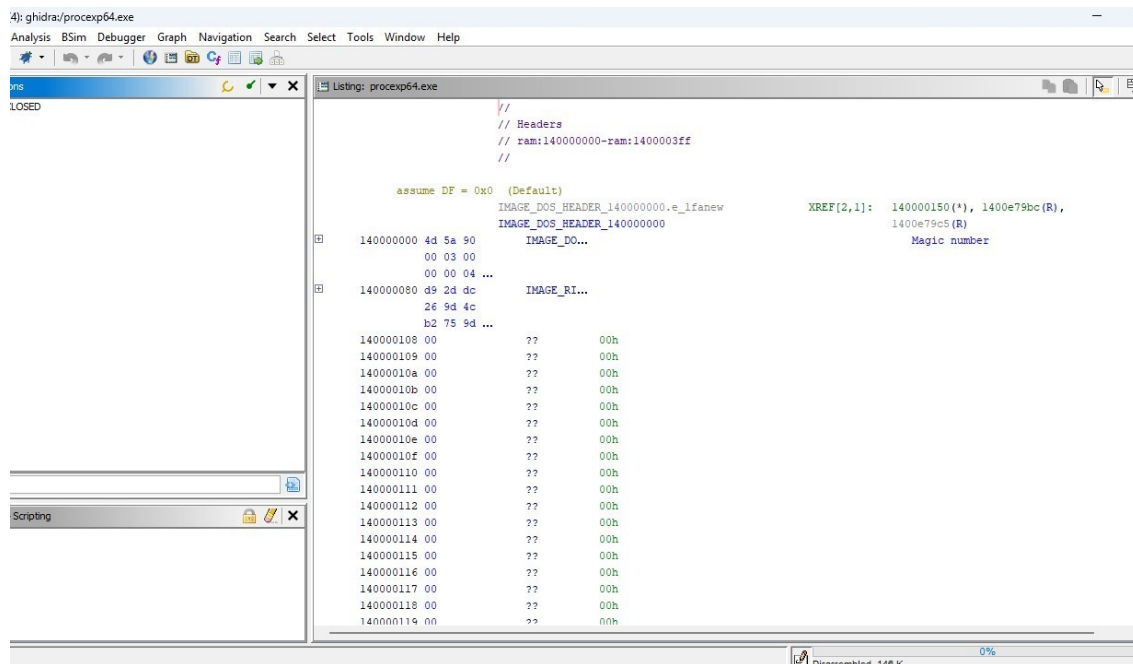


Figure 2: Ghidra Code Browser interface displaying the disassembly after Auto-Analyze

## Step 2: String Extraction and Import Analysis

1. Navigate to Window > Defined Strings to extract hard-coded ASCII and Unicode strings from the binary.
2. In the Symbol Tree pane on the left, expand the Imports directory to identify the external shared libraries the binary relies on (e.g., libc.so.6).

|           |                                           |                                        |          |        |         |
|-----------|-------------------------------------------|----------------------------------------|----------|--------|---------|
| 14000004d | db 21h (byte[64]e_progra...               | "!This program cannot be run in DOS... | string   | 45     | true    |
| 140000228 | IMAGE_SECT...                             | IMAGE_SECTION_HEADER                   | ".text"  | string | 6 false |
| 14000024f | SectionFlags IMAGE_SCN_C...               | ".rdata"                               | string   | 8      | true    |
| 140000277 | IMAGE_SECT...                             | SectionFlags IMAGE_SCN_C...            | ".data"  | string | 7 false |
| 1400002a0 | IMAGE_SECT...                             | IMAGE_SECTION_HEADER                   | ".pdata" | string | 7 true  |
| 1400002c7 | SectionFlags IMAGE_SCN_C...               | ".rsrc"                                | string   | 7      | false   |
| 1400002ef | SectionFlags IMAGE_SCN_C...               | ".reloc"                               | string   | 8      | true    |
| 14004f99c | MOV dword ptr [RBX + 0x4...               | "C@\\n\\n\\n"                          | string   | 6      | false   |
| 14004fa9a | MOV dword ptr [RBX + 0x4...               | "CD%\$#"                               | string   | 6      | false   |
| 14004fa0a | MOV dword ptr [RBX + 0x6...               | "C\\PPP"                               | string   | 6      | false   |
| 14005559d | MOV dword ptr [RBX + 0x4...               | "CL\\n\\n\\n"                          | string   | 6      | false   |
| 1400555a4 | MOV dword ptr [RBX + 0x5...               | "CP%\$#"                               | string   | 6      | false   |
| 14007ee68 | MOV qword ptr [RSP + 10c...               | "D\$dI"                                | string   | 6      | false   |
| 14009a50e | MOV R10,0x747865742e                      | ".text"                                | string   | 6      | false   |
| 14009a522 | MOV R11,0x617461642e                      | ".data"                                | string   | 6      | true    |
| 14009a60d | MOV R10,0x7373625f7472                    | ".rt_bss"                              | string   | 7      | false   |
| 1400bf86e | CMP qword ptr [DAT_14019...               | ".l9=;p\\y"                            | string   | 7      | false   |
| 1400c0e01 | LAB_1400c0... CMP qword ptr [DAT_14019... | ".l95PZ\\y"                            | string   | 7      | false   |
| 1400c79ef | JZ LAB_1400c7a20                          | ".t\\H;="                              | string   | 6      | false   |
| 1400cfa9e | CMP qword ptr [DAT_1401a...               | ".H95s;\\y"                            | string   | 7      | false   |
| 1400d1458 | LAB_1400d1... CMP EDI,dword ptr [DAT_1... | ".;=nG\\n"                             | string   | 6      | false   |
| 1400dc11f | MOV dword ptr [RSP + 10c...               | ".D\$ \\y"                             | string   | 6      | false   |
| 1400f02d9 | MOV dword ptr [RSP + 10c...               | ".D\$0f\\n"                            | string   | 6      | false   |
| 1400f02e6 | MOV dword ptr [RSP + 10c...               | ".D\$4p\\n"                            | string   | 6      | false   |

Figure3:ExtractionofDefinedStringsandidentificationofsystemImports

### Step3:Function Decompile

1. IntheSymbolTreepane,expandtheFunctionsdirectory.
2. Locateandselecttheentry function.
3. ObservetherawAssemblyinstructionsinthecenterListingwindow,andanalyzethe translated C-like pseudocode in the right-hand Decompile window.

```

2 void FUN_140001050(void)
3
4 {
5     DAT_14017d1c8 = FUN_1400edd8c(L"Ai
6     FUN_1400ede10((LPVOID)0x0);
7     atexit(FUN_14011e720);
8     return;
9 }

```

Figure4:DecompiledC-likepseudocodeoftheentryfunction

#### Step4:ExecutionFlowVisualization(MainFunction)

1. Due to the binary being stripped, the original main function name was removed. In the Decompiler window for the entry function, identify the first parameter passed to `libc_start_main`(e.g., `FUN_00104c00`).
2. Double-click `FUN_00104c00` to navigate to the actual main execution logic of the program.
3. With `FUN_00104c00` selected, navigate to **Window>Function Graph**.
4. Analyze the visual block diagram that illustrates the complex logical execution paths, decision branches (green and red arrows), and loops within the core code.



Figure5: Visual Function Graph illustrating the complex execution flow of the stripped main function (FUN\_00104c00)

## 5. OBSERVATIONS/FORENSIC FINDINGS

- **Code Structure:** Ghidra successfully disassembled the ELF binary, revealing the core architecture, data sections, and metadata headers.
- **Strings & Imports:** The extraction of defined strings and imports confirmed the binary's reliance on standard system libraries (libc.so.6), consistent with a benign Linux utility. In a malware context, this step would reveal imported networking, evasion, or encryption libraries.
- **Decompilation:** The Decompiler successfully translated the low-level assembly of the entry function into an understandable C-code format, highlighting the `libc_start_main` execution call.
- **Flow Analysis:** The Function Graph provided a clear, node-based visualization of the function's structural logic, simplifying the analysis of complex execution paths and potential infinite loops.
- **Stripped Binary Analysis & Symbol Resolution:** The binary had its symbol table stripped (.symtab removed), obscuring original developer function identifiers and variable names. Forensic examination of the execution entry vector resolved the actual main routine (FUN\_00104c00) by tracing the primary pointer passed into the `libc_start_main` parameter list.

## RUBRICS

| Criteria & Marks Assigned                              | Mark Allotted | Mark Awarded |
|--------------------------------------------------------|---------------|--------------|
| 1. GitHub Activity & Submission Regularity             | 3             |              |
| 2. Application of Forensic Tools & Practical Execution | 3             |              |
| 3. Documentation & Reporting                           | 2             |              |
| 4. Engagement, Problem-Solving & Team Collaboration    | 2             |              |
| Total                                                  | 10            |              |

## **6. RESULT**

Ghidra was successfully utilized to disassemble, decompile, and analyze the target binary code. Through automated analysis, string inspection, API import tracking, and decompiler evaluation, the binary's underlying structure and behavioral logic were effectively interpreted.